Friedrich-Alexander-Universität Erlangen-Nürnberg

FRIEDRICH-ALEXANDER
UNIVERSITÄT
ERLANGEN-NÜRNBERG

Internship Report

VAE-ry Ordinary Audio Synthesizer: Exploring Variational Autoencoders For Waveform Synthesis

submitted by

Purushotham Koduri

submitted

February 12, 2025

Supervisors

Johannes Zeitler

Prof. Dr. Meinard Müller

AUDIO LABS

International Audio Laboratories Erlangen

A joint institution of the
Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU) and
the Fraunhofer-Institut für Integrierte Schaltungen IIS.

Fraunhofer IIS

Abstract

In this project, we use autoencoders and variational autoencoders (VAEs) for the analysis and synthesis of audio waveforms. We begin with creating our own input audio data by generating basic wave forms like sine, square, triangle, and saw-tooth defined by shape and period, The data is used to train the models in a very controlled scenario. We use VAEs with a modified loss function, defined as a convex combination of the reconstruction loss and the Kullback-Leibler divergence. This small modification allows us to meaningfully give preference to either better reconstruction quality or more continuous latent space.

Further investigation is conducted on the model's capability to interpolate between different waveforms in both period and shape. To evaluate the performance of the models, we define metrics to measure the quality of reconstructions across different models, later we also evaluate the model interpolations in the frequency domain.Finally, we discuss why the models' performance significantly declines with more complex input dataset and propose further ideas to overcome this, in the process highlighting the challenges faced when dealing with more complex audio data, and also laying the foundation for further work.

Contents

1 Theoretical Foundations

We begin by introducing the basics of neural networks. We also formally describe autoencoders and variational autoencoders (VAEs). The loss functions used are discussed, and their derivations are provided. The process of controlling different components of the loss functions using a single parameter is illustrated. The model architecture used in most of the experiments in this project is also described.

1.1 Autoencoders

An Autoencoder(AE) [8] is a special case of a feed forward neural network [7], consisting of three core components, the encoder, the decoder and the latent space representation or the bottleneck. The encoder $z = f(x)$ compresses the input x to a lower dimensional representation and the decoder $\hat{x} = g(z)$ tries to reconstruct the input from this latent space representation denoted by $\hat{x}$, here both f, g are NNs. Essentially, the autoencoder tries to learn an approximation of the input identity. The learnable parameters of the encoder and decoder, W (the weight matrices) and b (the biases), are jointly optimized with an MSE-based loss function, which is given by [1]:

$$\text{MSE}_{W,b}(x, \hat{x}) = \frac{1}{n} \sum_{i=1}^{n} (x_i - \hat{x}_i)^2 \tag{1.1}$$

1.2 Variational Autoencoders

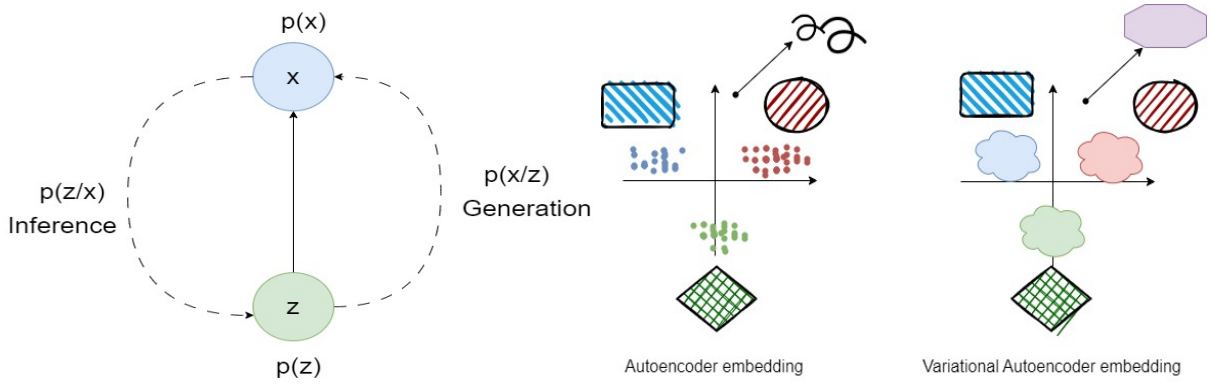
Autoencoders often create an unstructured latent space, where the learned embedding vectors lack meaningful relationships. To address this, Variational Autoencoders (VAEs) [9] describe the latent space probabilistically. This approach allows us to sample from the latent space to generate new data.

Consider a latent variable z that generates the input data x. To understand z, we want to compute $p(z \mid x)$, which allows us to generate new data by sampling from $p(x \mid z)$.

1.2.1 VAE Loss function

The problem with the above setup, however, is that $p(z \mid x)$, parameterized by θ, is intractable. This is denoted as $p_\theta(z \mid x)$, so we approximate this distribution with another tractable distribution $q(z \mid x)$, parameterized by ϕ, denoted as $q_\phi(z \mid x)$. This allows us to perform approximate inference of the intractable distribution. We can use Kullback-Leibler (KL) divergence [8] to quantify the distance between the real and approximate distributions $D_{KL}(q_\phi(z \mid x) \parallel p_\theta(z \mid x))$ with respect to θ, and derive the following:

$$\log p_\theta(x) = D_{KL}(q_\phi \parallel p_\theta(z|x)) + \mathbb{E}_{q_\phi}[\log p_\theta(z, x) - \log q_\phi] \tag{1.2}$$



(a) VAE stochastically maps observed x-space to laten z space

(b) AE and VAE embeddings comapred

Figure 1: VAE latent space illustrated

The first term in 1.2 is the Kullback-Leibler (KL) divergence $D_{KL}(q_\phi \parallel p_\theta)$, which is always non-negative and zero if, and only if, $q_\phi(z \mid x)$ equals the target distribution, which is the assumed posterior. The second term in 1.2 is the variational lower bound, also called the evidence lower bound (ELBO). In the context of Variational Autoencoders (VAEs), this second term can be further expanded to include the reconstruction loss. Specifically, the reconstruction loss measures how well the model can reconstruct the input data x from the latent variable z. We use Mean Squared Error (MSE) as the reconstruction loss function that we defined earlier 1.1

$$\mathrm{C}_{\mathrm{VAE}} = \mathbb{E}_{q_\phi(z|x)}[\mathrm{MSE}_{W,b}(\hat{x}, x)] + D_{KL}(q_\phi(z \mid x) \parallel p_\theta(z)) \leq \log p_\theta(x) \tag{1.3}$$

where $\hat{x} = p_\theta(x \mid z)$ represents the reconstructed data given the latent variable z. Minimizing this loss function maximizes the ELBO, thereby improving both the reconstruction accuracy and the quality of the latent variable distribution approximation.

1.2.2 Reparameterization trick

By minimizing the ELBO 1.3), we aim to maximize the lower bound of the probability for generating samples that are close to the true posterior distribution. Practically, this is done using SGD [1]. Since sampling is stochastic, we cannot backpropagate through this step, so we use a deterministic approximation for the random variable z. Specifically, we express z as $z = f(\phi, x, \epsilon)$, where ϵ is sampled from a unit Gaussian. We then shift this sample by the mean and scale it by the standard deviation of the latent distribution [3].

$$z \sim q_\phi(z|x^{(i)}) = \mathcal{N}(z; \mu^{(i)}, (\sigma^{(i)})^2, I) \tag{1.4}$$

$$z = \mu + \sigma \odot \epsilon, \text{ where } \epsilon \sim \mathcal{N}(0, I) \tag{1.5}$$

where μ and σ are outputs of the encoder component, I is the covariance matrix, which in this case is an identity matrix as there is no assumed relationship between the two latent dimensions and $\epsilon \sim \mathcal{N}(0, 1)$ is an auxiliary variable drawn from a standard normal distribution [6].

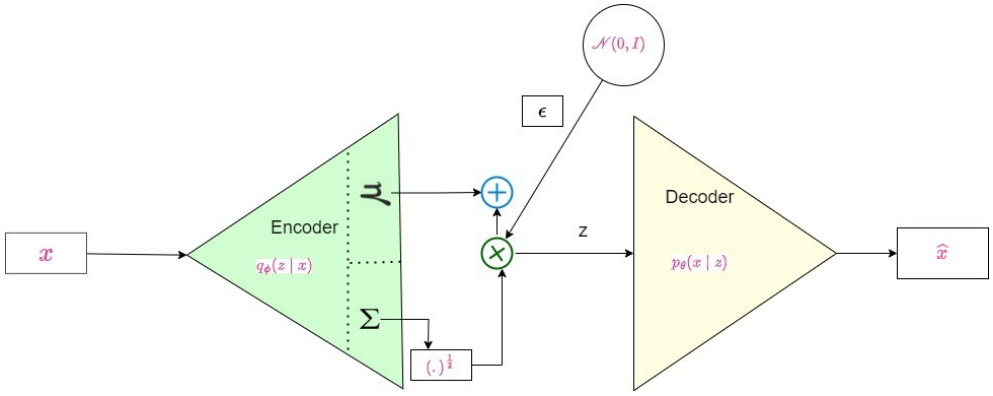


Figure 2: DNNs as VAE encoders and decoders [11]

1.2.3 VAE Architecture

By using the reparameterization trick, we handle the randomness of the posterior distribution through a separate variable ϵ. This approach externalizes the randomness, which allows us to back propagate through every step of the network.

It is common to assume that both the prior distribution $p(z)$ and the approximate posterior $q(z \mid x)$ are multivariate Gaussian distributions. This assumption allows us to compute the Kullback-Leibler (KL) divergence analytically. For Gaussian distributions, the KL divergence between the approximate posterior $q(z \mid x)$ and the prior $p(z)$ is given by:

$$\mathrm{KL} = -\frac{1}{2}\sum_{d=1}^{D}\left(1 + \log(\sigma_d^2) - \mu_d^2 - \sigma_d^2\right) \tag{1.6}$$

where μ_d and σ_d^2 are the mean and variance of the d-th dimension of the approximate posterior $q(z \mid x)$, respectively. This formula provides an explicit way to calculate the KL divergence for Gaussian distributions [5].

$$\mathrm{C}_{\mathrm{VAE}} = \mathrm{KL} + \mathrm{MSE}_{W,b}(x, \hat{x}) \tag{1.7}$$

It is therefore important to strike a balance between the two loss components, we can modify the loss function by adding another parameter β and express the two losses as a convex combination to achieve greater control over the loss function. For example, we can say that an autoencoder is a special case of a VAE, as we can easily switch from a VAE to a plain autoencoder by setting the value of β to 0 in 1.8.

$$\mathrm{C}_{\mathrm{VAE}} = \beta * \mathrm{KL} + (1 - \beta) * \mathrm{MSE}_{W,b}(x, \hat{x}) \tag{1.8}$$

2 Experiments With Simple Waveforms

The experiments section provides overview on the input data, experimental results, description of model architecture that was used to generate the results, After describing the results we evaluate the quality of the results and finally discuss insights generated during the experiments.

2.1 VAE Network Layout

The encoder compresses the input from 512 dimensions down to a 2-dimensional latent space, where the mean and log variance are calculated. The decoder reconstructs the data from this latent representation back to 512 dimensions. The hyperbolic tangent (tanh) activation function is applied after each linear layer to introduce non-linearity.

Table 1: VAE Architecture Overview (Total Parameters : 87,460)

Component	Layer (Operation)	Input Size	Output Size	Parameters
Encoder	Linear (Input Layer)	512	128	32,896
	Linear (Hidden Layer 1)	128	64	8,256
	Linear (Hidden Layer 2)	64	36	2,340
Latent Space	Linear (Mean)	36	2	74
	Linear (Log Variance)	36	2	74
Decoder	Linear (Latent to Hidden)	2	36	108
	Linear (Hidden Layer 1)	36	64	2,368
	Linear (Hidden Layer 2)	64	128	8,320
	Linear (Output Layer)	128	512	33,024

2.2 Data

Our training dataset consists of audio waveforms differing in shape and frequency. The waveforms are categorized into four distinct shapes: sine, square, triangle, and sawtooth waves. The frequency of these waveforms ranges from 20 Hz to 20,000 Hz, covering the entire audible spectrum.

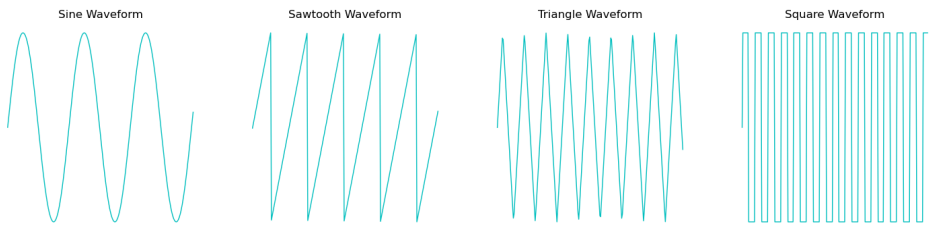


Figure 3: Examples of different waveform shapes used in the training data.

The waveforms can also be represented in the frequency domain by applying the Fast Fourier Transform (FFT) [10] to each sample. Consider the waveforms as a matrix, where each row

corresponds to a separate sample, and each column represents a data point within that sample. By applying the FFT to each row, we transform the time-domain data into its frequency components, while preserving the original matrix dimensions. As a result, each row in the frequency representation shown in Figure 4 represents the frequency spectrum of an individual waveform, with the horizontal axis corresponding to the frequency bins, the resultant plot can be

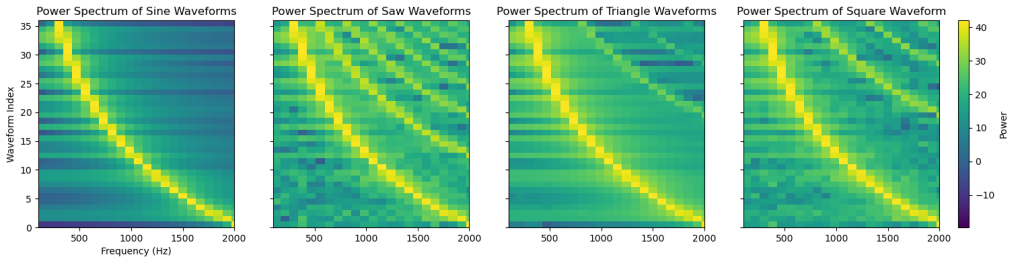


Figure 4: Frequency representation Comparison of Sine, Square, Triangle, and Sawtooth Waveforms.

2.3 Results

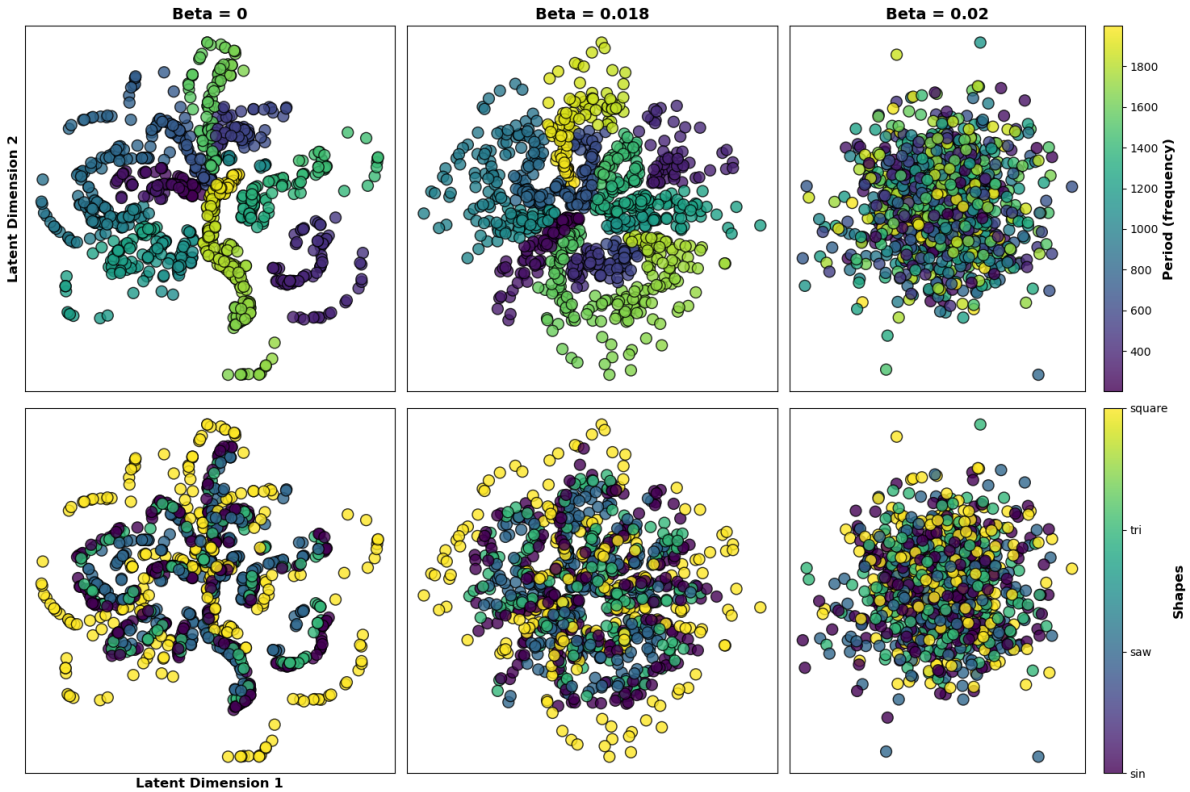


Figure 5: Embeddings generated by VAEs of different β values

In this section, we discuss the results and properties of three different VAE models we experimented

with. We start with a general investigation of the models. Then, we explore the interpolations produced by each model. Finally, we evaluate these interpolations in the frequency domain.

We start by visualizing the embeddings of the three models using scatter plots. To do this, we encode the entire training dataset and then plot all the encoded samples in a 2-dimensional space, where each axis represents one dimension of the 2D latent space The plots are colored based on period and shape. The color scale on the top row maps the period values, with darker colors representing lower frequencies and brighter colors representing higher frequencies. The bottom row visualizes the same latent space embeddings, but the color corresponds to different shapes. The color bar on the right indicates different shapes: square, triangular, sawtooth, and sine waves. The models are evaluated using three different β values. Setting β (refer to 1.8) to 0 removes the influence of the KL term, reducing the network to a regular autoencoder. In this case, the embedding space becomes sparse, with many empty areas in the plot. This suggests already that in such a sparse space, interpolations may not be possible everywhere. With the β as 0.018 we see a denser space, and we see a much better separation for both period and shapes, for periods especially we see nice clusters for different frequencies. Finally, we trained a model with β value of 0.02 with high importance on KL term, this results in a space which is closest to the posterior of choice, but this also means that far less emphasis on the reconstruction of the original data compared to the other models.

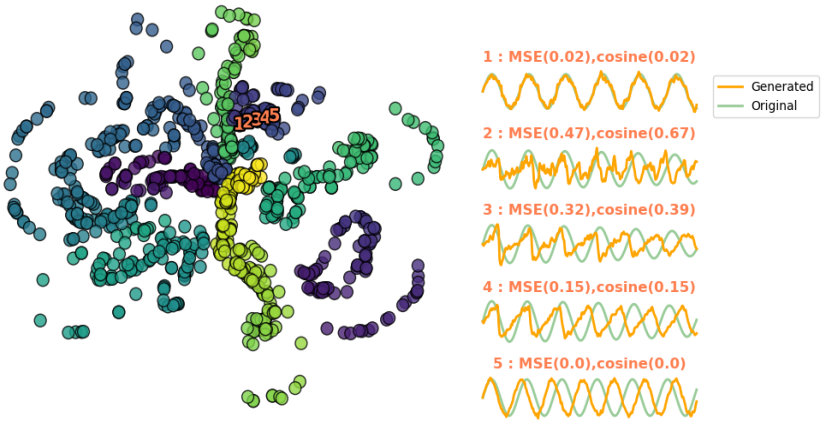
2.4 Interpolation between periods

In this section, we perform linear interpolation between two sine waves with periods of 500 and 600, respectively. We first encode the original sine waves to obtain their embeddings, and then linearly interpolate between these embeddings to create intermediate representations. These interpolated embeddings are then decoded back into waveforms. Ideally, the decoded waveforms should match the reference sine waves. To measure the similarity between the decoded and reference signals, we use the mean squared error (MSE) 1.1 and cosine similarity [2].

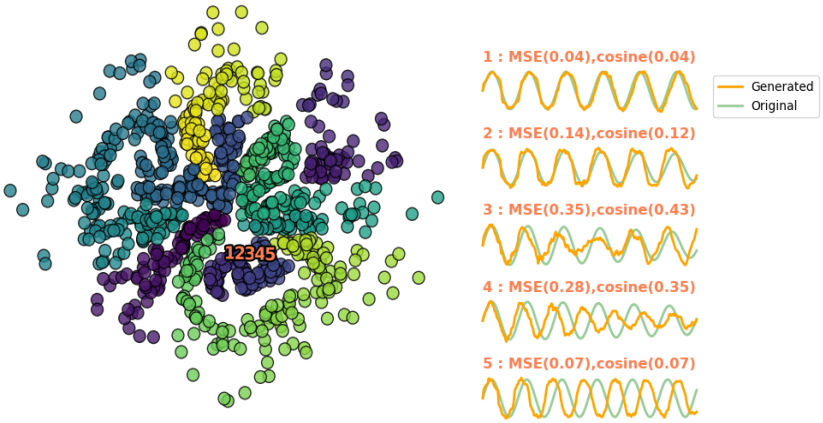
Cosine Similarity: We wanted a metric other than MSE which would not be influenced by the magnitude of the vector too much, on this end we use cosine similarity, which measure the cosine angle between two vectors. A cosine similarity of 0 indicates that the vectors are identical (pointing in the same direction), while a cosine similarity of 1 means the vectors are orthogonal. A cosine similarity of 2 implies that the vectors are completely dissimilar and point in opposite directions.

$$\text{CS} = 1 - \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \tag{2.1}$$

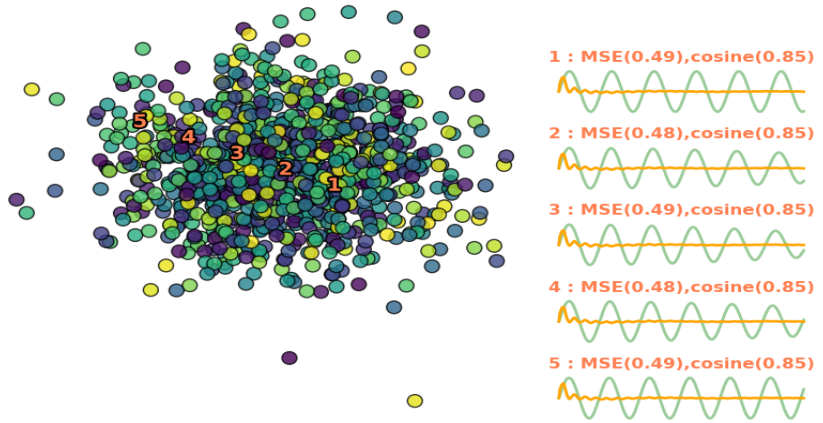
Figure 6a shows the interpolations for a VAE with $\beta = 0$. The original reference waves are displayed on top, along with the decoded interpolations. We also plot the embeddings in the latent space. Since there is no KL component in the training loss, in other words the network is equivalent to an autoencoder, the performance remains consistent across repeated experiments. The metrics show that the network performs well when reconstructing the original waves with periods of 500 and 600. However, performance drops for the intermediate interpolations.



(a) $\beta = 0$



(b) $\beta = 0.018$



(c) $\beta = 0.02$

Figure 6: Linear interpolations between periods for different values of β

β value	Average Mean Square Error	Average Cosine Similarity
$\beta = 0$	0.2862	0.3050
$\beta = 0.018$	0.2486	0.2868
$\beta = 0.02$	0.4900	0.8499

Table 2: Average Cosine similarity and Average Mean square error for the experiment

Next, we repeat the experiment with $\beta = 0.018$ in the figure 6b, aiming to balance the tradeoff between KL and $\text{MSE}_{W,b}$. In this case, the network gives similar performance to the $\beta = 0$ setting for the original reconstructions. Additionally, there is a slight improvement in performance for the intermediate interpolations. However, this improved performance comes with a caveat. Since a VAE samples from a structured latent space, we get different results each time we repeat the experiment. While the variations in quality aren't drastic, they are significant enough to mention. One significant observation to make is that the intermediate interpolations are less noisy than the autoencoder counterpart, suggesting the network managed to learn meaningful embeddings that relate well with each other.

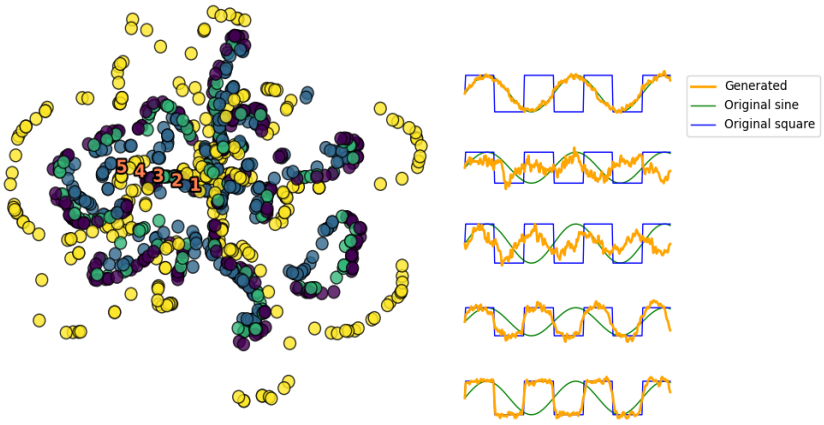
In Figure 6c, we increase β to 0.02. At this value, the network heavily prioritizes the KL term over the reconstruction term. As a result, it fails to learn meaningful features for reconstruction and instead produces outputs that resemble Gaussian noise, regardless of the input.The interpolation in latent space becomes random, and the mean squared error (MSE) remains nearly constant for every input. This happens because, in trying to optimize the reconstruction loss, the model learns only a general feature of the entire training dataset. The best way to minimize reconstruction error in this scenario is for the model to output the mean of the entire training data.

We conducted the same experiment using periods from 200 to 1000, interpolating at intervals of 100. Each interval was repeated 100 times, and we calculated the average mean square error (MSE) and average cosine similarity. The results are summarized in Table 2. The VAE with $\beta = 0.018$ consistently performed better in both MSE and cosine similarity compared to the others networks.

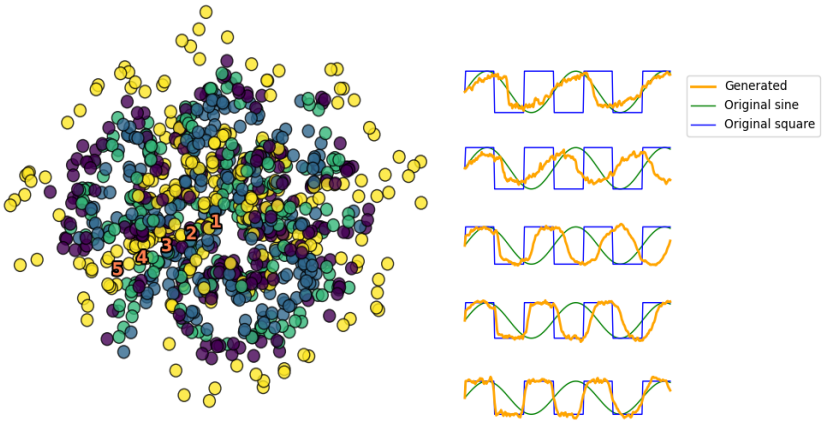
2.5 Interpolation between shapes

In the previous experiment, we sampled from the VAEs for periods, but now we do something similar for shapes, However, the networks did not really see how intermediate shapes between each shape would look like whereas in period, between 200 and 300 the training data consisted of many intermediate samples in that range, In this experiment we now linearly interpolate between shapes and try to notice how our networks handle these sudden shifts.

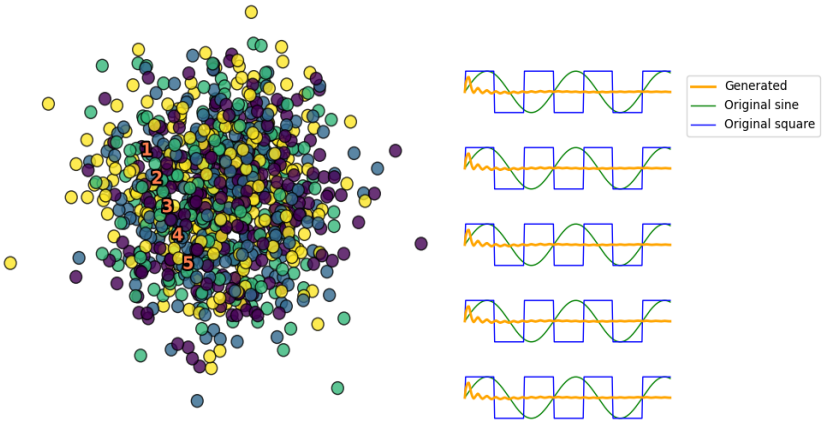
We perform this experiment by first defining a sine wave of period 300 and a square wave of period 700 respectively, we then generate embedding vectors for these two inputs, we then linearly interpolate between these two embeddings and finally decode these embeddings to generate output waves.



(a) $\beta = 0$



(b) $\beta = 0.018$



(c) $\beta = 0.02$

Figure 7: Linear interpolations between shapes for different values of β

From the figure 7a we see the network with $\beta = 0$ does reasonably well with generating the output waveform for both the original inputs, but for the interpolations themselves the outputs are very noisy and one cannot tell that these waves are supposed to be intermediate waves between the two inputs. In contrast, 7b we see for the VAE with $\beta = 0.018$ that the intermediate waves are smoother and the shapes of the waves look as though they are indeed the intermediate waves between the two input vectors. Finally, the network with $\beta = 0.02$ the outputs generated are exactly the same, the Gaussian like wave which owes most of its structure to the mean of the entire training sample data, making the interpolations indistinguishable from the input waves.

This experiment shows how embedding relationships impact the generated outputs. We conclude that a VAE with a well-chosen β value, such as $\beta = 0.018$, generates more meaningful interpolations that translate into smoother and more accurate decoded waveforms.

2.6 Assessing reconstruction quality in spectral domain

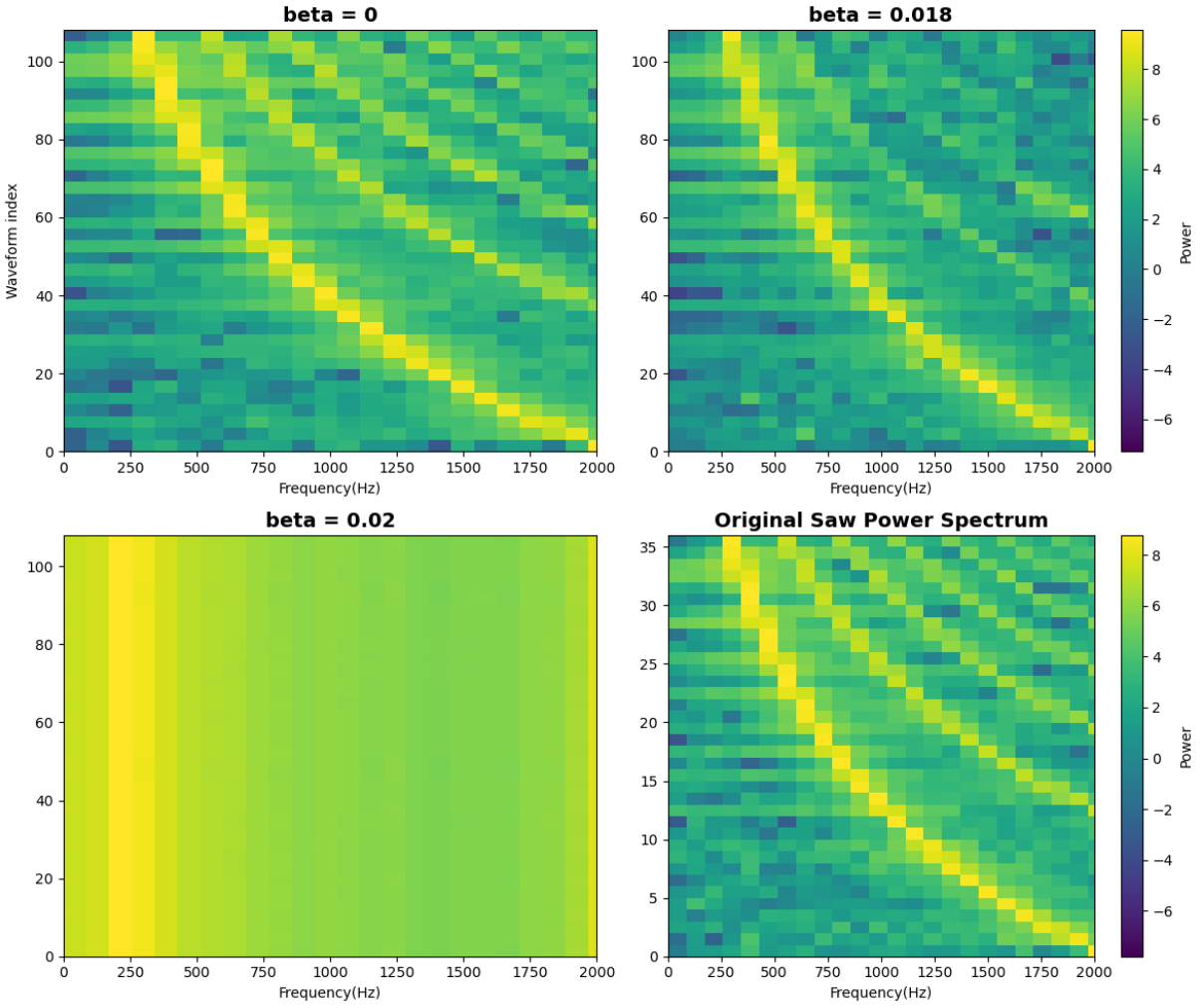


Figure 8: Spectrograms from the original input and reconstructed outputs of different networks.

Until now, we evaluated the interpolations from all the three networks using metrics or by

noticing how noisy the transitional interpolations are, all of this however is in the time domain, we have seen 4 that our input data has interesting spectral patterns especially when viewed in Spectrogram as shown earlier, this adds another dimension to the evaluation of our three models.

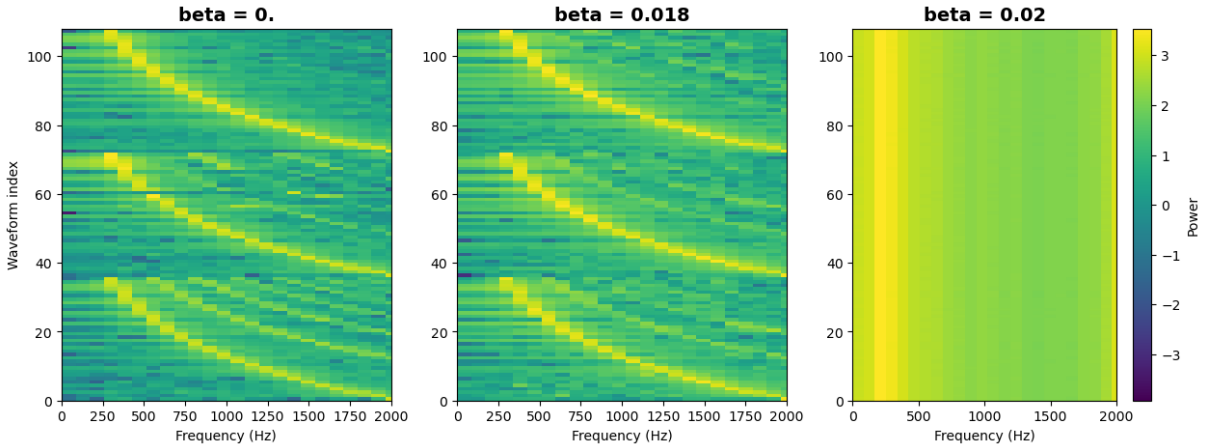


Figure 9: Spectrograms of the interpolated audio between sine and saw waveform for our three networks.

Out of all the different types of waveform available for us, saw waves has the highest amount of partials, so we take a closer look at the spectrogram of the saw wave and its decoder reconstructions from our three networks in 8. On the vertical axis, we have the individual waveforms, which are all saw shape but of different periods, identified by their index. On the horizontal axis we have limited the frequency range from 0 to 2000, this is because our training data ranges from 200 to 2000, this can further be illustrated in the original saw power spectrum plot.

We will limit ourselves to qualitative analysis for this evaluation process, as we can make out the differences in the spectrogram without any need of an external metric, but just by looking at the plots themselves. As expected the best reconstruction performance is from the autoencoder, the VAE generates decent reconstruction, but the resolution for the harmonics is not as good as the plain encoder, the final spectrogram where the $\beta = 0.02$ just manages to plot the frequency response of the average waveform over the entire training dataset.

Next, we will extract embedding vectors from both a sine wave and a saw wave. We'll then average these vectors and feed the result to the decoder, which effectively creates a linear interpolation between the sine and saw waves. This approach gives us more insight into the power spectrum of the interpolations, rather than just reconstructing the original inputs.

In Figure 9, we observe three stacked spectrograms. The top and bottom subplots show the sine and saw waves, while the middle one represents a linear interpolation between the two. Although the autoencoder reconstructs the original sine and saw waves well, the middle interpolation only captures the fundamental frequency, with almost no harmonic components. On the other hand, the VAE with $\beta = 0.018$ retains some harmonic information, which aligns with the expectation that the interpolation should resemble a mix of sine and saw waves.

3 Single Note Database (SNDB)

To apply the methods we learned so far, we decided to train a network on a more realistic dataset, we now train our networks on a subset of the single note database(SNDB) [4] containing 504 total audio samples from 7 different instruments: piano (PIA), violin (VLN), violincello (VLC), trumpet (TRP), oboe (OBO), clarinet (CLA) and flute (FLU). There are three different dynamics (pianissimo, mezzo, and forte) with midi pitches [**?**] ranging from 60 to 83, i.e., two whole octaves. The original plan of the experiment was to start with building a plain autoencoder by having, $\beta = 0$ and then increase the beta value after achieving satisfying reconstructions of the input audio.

The encoder compresses the input from 88200 dimensions down to a 32-dimensional latent space. Again the hyperbolic tangent (tanh) activation function is applied after each linear layer to introduce non-linearity. The model is clearly bigger compared to our previous networks, this was done for the more complicated nature of the data.

Table 3: SNDB VAE Architecture Overview (Total Parameters: 182,124,616)

Component	Layer (Operation)	Input Size	Output Size	Parameters
Encoder	Linear (Input Layer)	88200	1024	90,318,824
	Linear (Hidden Layer 1)	1024	512	524,800
	Linear (Hidden Layer 2)	512	256	131,328
	Linear (Hidden Layer 3)	256	128	32,896
	Linear (Hidden Layer 4)	128	64	8,256
Latent Space	Linear (Mean)	64	32	2,080
	Linear (Log Variance)	64	32	2,080
Decoder	Linear (Latent to Hidden)	32	64	2,112
	Linear (Hidden Layer 1)	64	128	8,320
	Linear (Hidden Layer 2)	128	256	33,024
	Linear (Hidden Layer 3)	256	512	131,584
	Linear (Output Layer)	512	88200	525,312

The table above outlines our model's architecture. Using this setup, we trained the network for approximately 120 epochs. This number was chosen after testing various options, and we continued increasing the epochs as long as the reconstruction loss kept decreasing. In practice, the loss plateaued around 120 epochs.

To evaluate the network's performance, we generated a spectrogram for each instrument by plotting the C major scale. This was done by auto encoding the dataset's waveforms for each note in the seven-note scale, concatenating them, and generating the corresponding spectrogram. We repeated this process for all seven instruments in our dataset, and the resulting spectrograms are shown in Figure 10.

The horizontal axis represents time in seconds, while the vertical axis shows MIDI pitch numbers. We chose MIDI numbers instead of frequency in Hertz because human perception of frequency is logarithmic [**?**], making this representation more meaningful. One noticeable observation is the absence of energy in MIDI pitches below 60, which reflects the limits of our training data, as it only contains MIDI pitches ranging from 60 to 83. However, we do observe energy in pitches

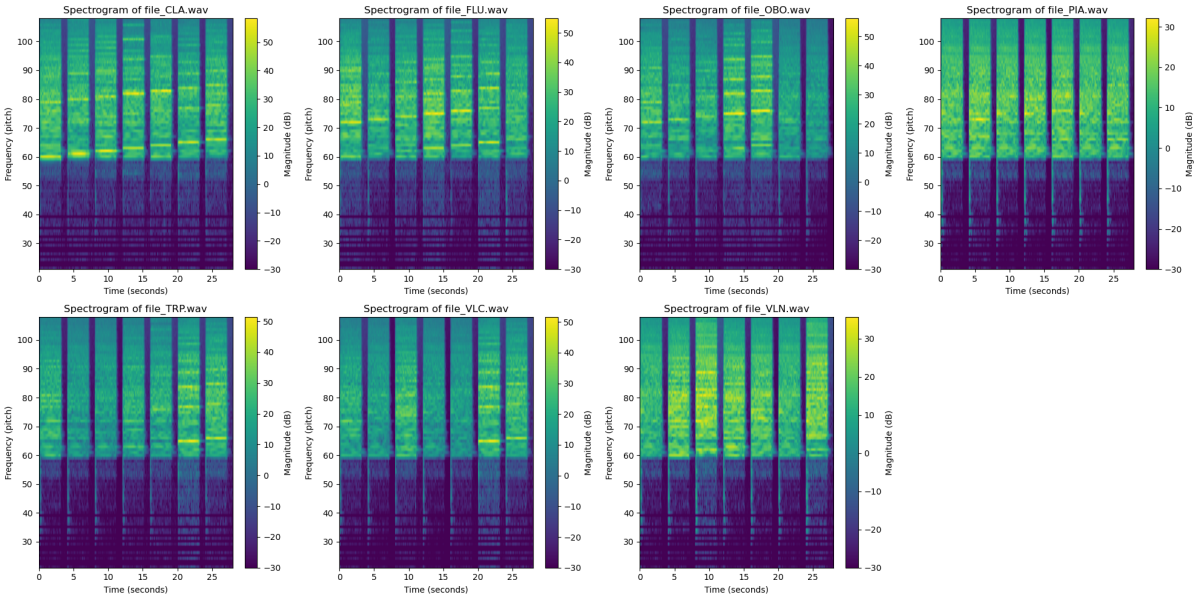


Figure 10: Spectrograms for the decoded audio for all the seven instruments

above 83, likely due to harmonic overtones from the notes.

Distinct vertical gaps with no energy correspond to the silence at the start and end of the audio samples. While this could have been processed out, it was not a priority for this experiment. Additionally, horizontal lines in the lower pitch range are a result of the frequency-to-MIDI conversion process. Since lower pitches are sampled across a smaller range of frequencies, there are fewer available sampled frequencies, which leads to more visible quantization effects at lower pitches [**?**].

Wind instruments, such as the clarinet and flute, show significantly better reconstructions compared to other instruments. This can be seen in their spectrograms, while instruments like the piano and violin exhibit poorer reconstructions. This is due to the complex harmonic structures of instruments like the piano and violin, whereas the clarinet and flute are acoustically closer to a simple sine tone.

4 Conclusion

Variational Autoencoders are powerful tools, they are the first step towards generative modelling. When built with a proper balance between the two key loss components, we can design meaningful embeddings, which have the ability to not only generate the input data but also generate entirely new examples that were not seen in the training data. However, it's important to remember that working with complex input data often requires large amounts of training data. In such cases, more advanced neural networks may be needed, which in turn demand more powerful hardware to handle the increased complexity.

References

[1] C. Bishop, *Pattern Recognition and Machine Learning*, vol. 16, Springer, 01 2006, pp. 140–155.

[2] P. Dangeti, *Statistics for Machine Learning*, Packt Publishing, 2017.

[3] C. Doersch, *Tutorial on variational autoencoders*, 2021.

[4] E. Feichtner and B. Edler, *Description of the single note database sndb.* `https://publica.fraunhofer.de/handle/publica/407570`, 2018. -1.

[5] G. Fyffe, *There and back again: Unraveling the variational auto-encoder*, 2021.

[6] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, *beta-VAE: Learning basic visual concepts with a constrained variational framework*, in International Conference on Learning Representations, 2017.

[7] K. Hornik, M. B. Stinchcombe, and H. L. White, *Multilayer feedforward networks are universal approximators*, Neural Networks, 2 (1989), pp. 359–366.

[8] D. P. Kingma and M. Welling, *Auto-encoding variational bayes*, in Proceedings of the International Conference on Learning Representations (ICLR), 2013.

[9] D. P. Kingma and M. Welling, *An introduction to variational autoencoders*, Foundations and Trends in Machine Learning, 12 (2019), p. 307–392.

[10] A. V. Oppenheim, R. W. Schafer, and J. R. Buck, *Discrete-Time Signal Processing*, Prentice Hall, Upper Saddle River, NJ, USA, 2nd ed., 1999. Chapter 10 covers the Short-Time Fourier Transform and other time-frequency representations.

[11] J. Zeitler, *Manifold learning of acoustic relative transfer functions*, 2021.